

Cahier des Charges Fonctionnel & Technique

— Plateforme MaSecure

Infrastructure de Règlement Social Automatisé

Statut : Document de Référence pour la Collaboration Externe

Date : Juin 2026

1. Présentation Générale et Vision du Projet

1.1. Problématique

Les tontines rotatives traditionnelles (épargne collective communautaire) en Afrique de l'Ouest souffrent d'opacité financière, de risques de fraude du trésorier humain, de difficultés de traçabilité des contributions et de conflits lors des retards de paiement ou des sinistres (décès, abandons).

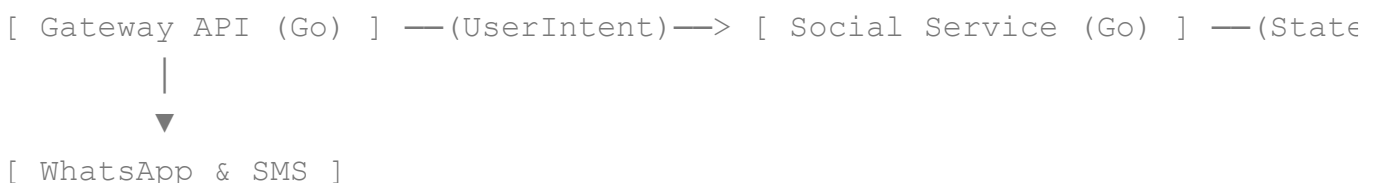
1.2. Solution MaSecure

MaSecure est un orchestrateur financier sécurisé et automatisé. Il remplace le trésorier humain par un **noyau applicatif déterministe** et un **coffre-fort de fonds neutralisé** logé dans une banque partenaire, accessible simplement via WhatsApp, SMS et application mobile.

La confiance repose sur des règles d'exécution strictes écrites dans le code et sur un journal d'événements cryptographiquement chaîné (ledger append-only).

2. Architecture Logicielle : Triple Isolation

Pour des raisons de sécurité absolue des transactions, MaSecure applique une architecture en triple isolation :



1. **Gateway Communication (API Gateway Go)** : Interface d'entrée publique (webhooks WhatsApp/SMS). Elle ne possède aucun droit financier, traduit les messages en intentions (`UserIntent`) et les transmet.
 2. **Service Social & Gouvernance (Go)** : Gère l'administration des groupes, les propositions de configurations, le recalage des ordres et l'enregistrement des votes.
 3. **Kernel Financier (Rust)** : Noyau souverain isolé. C'est le **seul** service possédant les clés d'API bancaires et émettant les ordres de virement (`PayoutCommand`) après vérification mathématique des invariants en base de données.
-

3. Règles Métier Fondamentales (Invariants)

3.1. Gestion du Fonds de Recouvrement

- **Optionnel** : Choisi à la création du groupe.
- **Financement** :
 - *Intégral* : Versé en une fois au démarrage.
 - *Progressif* : Versé en tranches régulières.
 - *Invariant de Sécurité* : Le fonds de recouvrement progressif doit être **complété à 100% avant le premier versement (payout n°1)** du cycle 1. Sinon, le système gèle l'exécution des paiements.

3.2. Gestion des Retards & Automate de Sanction

- **Jour J (Échéance de contribution)** : Si un membre manque son versement, le fonds de recouvrement comble le déficit pour payer le bénéficiaire à temps.
- **Cycle suivant** : Le membre défaillant passe en statut `QUARANTINE` . Un court délai de grâce (ex : < 1 semaine) lui est accordé pour rembourser le fonds.
- **Suspension** : Si le délai expire, le statut passe à `SUSPENDU` . Le membre perd immédiatement son droit de vote. Pour être réintégré, il doit payer l'avance + la pénalité + les cotisations cumulées.
- **Exclusion** : Après **3 contributions manquées consécutives**, le système déclenche la procédure d'exclusion définitive.

3.3. Gestion du Décès (Sinistres)

- **Membre Débiteur (a déjà reçu le pot) :** Si un successeur désigné ne reprend pas la place, le fonds de recouvrement absorbe la dette (dans sa limite de couverture).
- **Membre Créditeur (n'a pas encore reçu le pot) :** Si aucun successeur ne reprend le compte, le système exclut le membre, calcule le total de ses cotisations passées et effectue un **payout anticipé immédiat de remboursement** à la famille.

3.4. Gestion des Frais de Mobile Money

- Si le groupe a un fonds de recouvrement actif : les frais de virement Mobile Money sont payés par le fonds (le bénéficiaire reçoit 100% de sa cagnotte).
 - Si le groupe n'a pas de fonds actif : les frais de virement sont déduits de la cagnotte (le bénéficiaire reçoit le pot moins les frais opérateur).
-

4. Stack Technique et Environnement de Dev

- **Backend :**
 - *Kernel* : Rust (déterminisme, sûreté mémoire).
 - *APIs & Workers* : Go (légèreté, goroutines concurrentes).
 - *Message Broker* : NATS JetStream (propagation asynchrone des événements).
 - **Frontend :**
 - *Mobile Client* : Flutter (Dart) – performance native compilée AOT, idéal pour les smartphones Android d'entrée de gamme en Afrique de l'Ouest.
 - *Web Admin* : React.js + Vite + TypeScript (interface moderne, responsive et fluide).
 - **Persistence** : PostgreSQL 16 (ACID, Triggers de contraintes) + Redis (verrous d'idempotence).
 - **Dev Container** : Conteneur de développement basé sur **Ubuntu 22.04 LTS** officiel, isolant les runtimes Rust, Go, Java JDK, Flutter SDK et utilisant **Volta** pour épingler les versions de Node/npm.
-

5. Comment Collaborer sur le Projet

5.1. Démarrage Rapide

1. Ouvrir le dossier `platform` sous VS Code.
2. Lancer la commande de réouverture dans le **Dev Container** (VS Code détectera automatiquement la configuration dans `platform/.devcontainer/`).
3. Démarrer l'infrastructure locale (PostgreSQL, Redis, NATS) :

`make up`

4. Appliquer les migrations de base de données situées dans `platform/migrations/` :
- `008_governance.sql` : Tables de votes et propositions.
 - `009_schema_extensions.sql` : Colonnes de successeurs, frais, fonds progressif et triggers d'intégrité.

5.2. Compilation et Exécution des Services

- Compiler le Kernel Rust : `make build-kernel`
- Lancer le Kernel : `make run-kernel`
- Lancer l'API Gateway Go : `make run-api`
- Lancer le Dashboard Web : `cd frontend/web_dashboard && npm run dev`